The CIRIS Constitution: The Mathematics

The Mathematics, a volume of edition 1.0-rc11

Stewarded by Eric Moore

The Mathematics

What this is. Part 6, *The Substrate*: the canonicalisation boundary and the 1+4 line, the census, replication and the wholeness witness, the noise-floor model, the effective source count that gates aggregation, and the rules of the substrate stated with the arguments behind them.

Whom it binds. The implementer of a substrate: its census, replication, witness and aggregation functions are rules here, stated with RFC 2119 force, beside the arguments for them. A theorem among them binds by being true; the rules bind by conformance, like the rules of *The Wire*. The volume names the places where an argument is weaker than its statement.

What it promises. The substrate's rules with the arguments behind them: the noise floor as a checkable predicate, the effective count stated with its clamp, its ceiling and its exposed direction, the descent and replication rules, and the census. Two results carry mechanised witnesses, the effective-count discount and the census ballot model, and the register pins them; where an argument stands without a model, the text says so.

What it leaves out. The engineering: codec choices, chunk sizes and endpoint shapes are in *The Wire*. The ethics the mathematics serves are in *The Charter*.

Where the rest lives. The two witnesses are named in the register, which pins the repositories they live in. A reviewer who wants to kill a result should start from the register row, not from the prose.

Part 6 — The Substrate

6.1 The substrate: the census, replication, the witness, trust discovery

The substrate gives the federation three things that together serve M-1's durability of shared memory. **Measurement** — each node keeps an inventory of what it holds and how deeply (the paragraph below), and the mesh learns its own size and use from anonymous census returns (CC 6.1.9). **Replication** — an item its owner marks durable is held by a chosen number of nodes in its own cohort, chunk by chunk, and repaired when that number falls (CC 6.1.5); recovery is pulling the chunks from any holder. **Layered degradation** — the CC 5.3.3.2.4 `ChunkLayer` stack is stored per layer, so shedding the high-detail layers yields a clean coarse version of the same item; degradation is a property of the **layering**, and the fidelity metric `R` of CC 6.1.2 is what measures it. The rateless codec (RaptorQ, RFC 6330) serves **realtime audio and video delivery only** (CC 5.3.3.2) — loss recovery without retransmission — and is not a storage mechanism: nodes and relays never encode or decode stored content; they hold and forward sealed chunks. The substrate's wire shapes enter CEG as additive normative sections, each bound to CEG's existing trust, post-quantum, consent, replication, and anonymous-tier invariants.

The inventory (normative). Each node measures two things. *What it holds*: per scope and per plane, a count and a hybrid-signed Merkle root over the claims it holds — the wholeness witness of CC 6.1.1, which says *this is the whole I hold here at this epoch, and how many pieces it has. How deeply each item is held*: holders per item, the node's own share of the item's chunks, and the gap to the item's target (CC 6.1.5). What a node cannot see is withheld by design, not by omission: other cohorts' private holdings, anonymous content, and the identities behind a public count. Anything unclaimed within a claim's liveness window is **unknown, never zero** — a node MUST NOT read an absent or lapsed claim as *none* (CC 3.1.3.3). Every instrument is a pure fold over signed inputs — integer arithmetic, permutation-invariant, no local heuristic in the result — so two nodes holding the same inputs compute the same inventory, and a disagreement is detected by comparing witness roots, never argued from counts. What the mesh as a whole holds and does is learned only through the census (CC 6.1.9), never by reading another node's inventory. Measurement changes no behaviour by itself: the only path from measurement to action is the mesh-config plane (CC 6.1.5.1), which can relieve and never expand.

6.1.1 The wholeness witness: divergence detection

A `wholeness_witness:` object is a peer's **hybrid-signed Merkle root over a scoped projection of the claims it holds**, used to detect cross-peer / cross-region state divergence and to drive reconciliation. It is the federation's mechanism for *noticing* that two peers have drifted apart — a precondition for healing the drift (Integrity).

Namespace + scope (normative).

- **WW-naming** — the namespace is **`wholeness_witness:`**, never bare `witness:`. It is a *self-published state-root snapshot*, the **inverse** of the CC 5.3.1 transparency-log "witness" (an independent STH cosigner). A wholeness witness does **not** provide append-only / consistency / anti-equivocation guarantees and MUST NOT be substituted for CC 5.3.1.1 or the CC 5.3.3.3 per-stream STH.
- **WW-1** — the root MUST cover **only** the namespaces in the object's `claim_namespaces` field. A conformant peer is **not** required to witness everything it holds; coverage is per-

namespace opt-in.

- **WW-2 (anonymous/self exclusion — fail-secure)** — a wholeness witness MUST NOT include anonymous-tier records or `cohort_scope: self` local-tier rows (CC 5.3.2.4) as Merkle leaves, and `claim_namespaces` MUST NOT name such a namespace. Witnessing them would re-attribute deniable / self-private content to a stable `peer_id`. The leaf-walk MUST filter these out before computing the root.
- **WW-3** — `cohort_scope: family | community` content MAY be witnessed **only at the opaque `content_id`/manifest-digest grain**, never at a grain disclosing membership, plaintext, or `subject_key_ids` (CC 5.2 confidentiality preserved).
- **WW-4 (publication, and the inventory's size)** — a node publishes its witness roots on the constant tick (CC 5.4.5), never on a change, and compares each root it receives against its own for the same `(epoch_id, claim_namespace_set)` on receipt; that comparison is how the inventory (CC 6.1) detects disagreement. A witness carries `leaf_count`: the inventory's size for its namespaces at the resolution the privacy model allows (WW-2, WW-3) — a count over opaque leaves, never over members or plaintext.

Construction (normative).

- **Leaf order MUST be lexicographic** over leaf bytes (the CC 2.6.1.1.1 set-semantics rule). Any "either order as long as both peers agree" convention is **non-conformant** — it is the CC 2.6.1-class divergence hazard.
- The Merkle scheme is `leaf = SHA-256(leaf_bytes)`, `node = SHA-256(left ‖ right)`, odd-node duplication, `b"WW-v1-empty"` empty sentinel. **CEG does NOT adopt the RFC 6962 `0x00`/`0x01` leaf/node prefix here.** Rationale: the odd-node-duplication malleability is **not exploitable in this construction's uses, given one rule: the leaf list MUST be deduplicated as well as sorted — the CC 2.6.1.1.1 set rule, sorted AND unique — and a list with a repeated leaf is refused before hashing.** (A repeated id in `sources` is refused at descent and in the aggregate builder as `federation_malformed_row` / `aggregate_source_duplicate`, so `source_count` and `n_eff` always count distinct ids.) Without that rule the lists `[a,b,c]` and `[a,b,c,c]` share a root under odd-node duplication and neither signing nor full-list recomputation tells them apart; with it, the duplicated list is never a valid input, and the remaining uses are safe because (1) every wholeness witness and `member_commitment` (CC 6.1.2.1.1) root is **mandatorily hybrid-signed** — no consumer ever relies on an *unsigned* root; (2) `member_commitment` is verified by **recomputation from the full source-id list**, never by partial inclusion proofs against the bare root (malleability moot); (3) the CC 6.1.1 reconciliation Merkle-proof exchange (N4) is between **accountable, signed, equivocation-checked peers** — not third-party forgery of an untrusted root. **Caveat (normative):** any future use that relies on an **unsigned** root, or verifies **partial inclusion proofs against an untrusted root**, MUST first adopt the RFC 6962 `0x00`/`0x01` prefix + lone-node promotion (and re-cut the vectors). This is a **distinct construction from the CC 5.3.1 RFC 6962 log** (different algorithm + leaf domain); the two MUST NOT be cross-verified. Changing this scheme is a vector-invalidating wire change — not an editorial tweak.

Authority (normative).

- **N3** — a wholeness witness is a federation-tier attestation: hybrid PQC verified at ingest **and before** persistence to the witness corpus (CC 6.1.3); `compare_witnesses` MUST NOT run on an unverified witness.

- **N4 (equivocation)** — two validly-signed witnesses from the same `(peer_id, epoch_id, claim_namespace_set)` with different `merkle_root` are **non-repudiable equivocation proof**; the substrate MUST retain and surface them as a `hard_case:*` (CC 3.4 reserved-prefix candidate), never silently reconcile. Per-peer `epoch_id` MUST be anti-rollback-checked before `EpochBehind` is used as a reconciliation input (eclipse guard). Full cross-witness BFT MAY be deferred (CC 8.3) — but observed equivocation MUST NOT be discarded.
- **WW vs replication (the highest-value reconciliation)** — a wholeness witness is a **divergence detector that *triggers*** the CC 5.3.2.3 quorum-merge; it does **NOT** decide a merge and MUST NOT replace `monotonic_quorum` / `revision` anti-rollback for `revocation` / `partner_record` / `org_membership`. A `Divergent` verdict on those subject_kinds hands the decision to the CC 5.3.2.3 quorum-merge (quorum-ordered, anti-rollback) — otherwise a "recover from any holder" path could resurrect a revoked key (rollback). Detection and decision are deliberately separated: the witness sees, the quorum-merge rules.

6.1.2 The noise floor: retirement and the retained floor (normative)

One operation, not many. Revocation, retirement, capacity-eviction, scheduled expiry, and natural aging are **the same operation at different rates**: a **monotonic descent of an item's fidelity, driven by pressure**, toward and below a recoverability boundary called the **noise floor**. There is no separate "hard delete" primitive — *hard-delete is the fastest descent* (forced immediately below the floor); capacity-eviction is a slow one; aging is the slowest. All are equally valid instances of the one retirement operator. Collapsing these into a single axis is what lets the right-to-be-forgotten (Autonomy) and the durability of history (Integrity) share one mechanism instead of fighting.

The noise floor = the individual-recoverability boundary. An item is **above** the floor in a retained artifact iff it can be individually reconstructed from that artifact, **by the specified reconstruction procedure `R`, above a fidelity** ε; it is **below** the floor iff only its *contribution to a collective* survives — the item is **not individually recoverable by `R` above** ε. The predicate is deliberately **procedure-relative**, not an absolute information-theoretic claim: `R` and ε are **operator-tunable per media type** (a per-type reconstruction metric + threshold) and pinned by a CC 6.1.4 conformance vector the same way the other pinned values are, so "below the floor" is a **checkable** predicate rather than an unquantified assertion. Absent a declared `(R, ε)`, the pinned default `R` is the **layered-codec fidelity metric**: decode the retained CC 5.3.3.2.4 `ChunkLayer` set (each layer held whole, present or absent) and score the reconstruction against the source by the corpus-kind's per-type fidelity measure; the pinned default threshold is ε = `0.05`. **The retained layer set is not a fidelity measure** — it decides only *which layers decode at all*, and is the input to `R`, not `R` itself. In particular, once no holder retains the base layer (CC 6.1.5) only the signed envelope survives (the EnvelopeOnly tier): the residual fidelity there is exactly **0**, not ε. The floor does double duty and is the load-bearing normative quantity of this section: it is **both** the privacy boundary (a revoked item MUST be below it, for the declared `(R, ε)`, at every retained tier) **and** the durability floor (the collective blur sits below it, forever).

Adversary model — the scope of the below-floor MUST (normative). "Individually recoverable" is meaningless without saying *by whom, holding what.* The below-floor MUST above binds **relative to the pinned default adversary $\mathtt{A}_0$**: an adversary holding the retained artifact, the public codec parameters, and the tier's `AggregationMetaV1` metadata, and running exactly

the declared `R` — and holding **no** auxiliary knowledge of the item or of its co-members. Against A_0 the predicate is decidable by running `R` and measuring, which is what the CC 6.1.4 vector does; a declared `(R, ε)` MUST therefore be accompanied by its adversary class, defaulting to A_0. The MUST does **NOT** assert below-floor against an **unbounded** adversary holding side information — all other source members, or a prior over the subject. That limb is **unverifiable pending an instrument**: this document declares no adversary-relative advantage bound, no conformance vector measures one, and an implementation MUST NOT represent an A_0 below-floor result as an anonymity or erasure guarantee against a side-informed adversary. **Aggregation is not anonymization.** A candidate instrument exists and is cited as a *proposal only* — an LP pair-pinning gate, which decides by linear program whether a coarse statistic is already pinned by finer marginals; it has not been ported to this setting and nothing here rests on it. Until it is, the already-erased shortcut below stands on the source-diversity gate, never on an anonymity claim.

Ordinary retirement never forgets entirely — the memory pyramid. For witnessed content retired in the ordinary way, descent does not terminate at zero; prohibited content terminates at zero (CC 6.1.2.2), and anonymous content is outside the witness and must be able to disappear (WW-2). Two **mechanical** degradation operators (no reasoning, no agency — see below) carry it:

1. **Intra-object fade** — scalable/layered codec (CC 5.3.3.2.4 `ChunkLayer` spatial/temporal/quality), stored per layer: drop the high-detail layers → a clean coarse version of the same item.
2. **Inter-object aggregation** — *a picture of a thousand pictures*: tile / downsample / statistically composite **N → 1**. Recursed, this builds a pyramid (mipmap) of history: recent strata high-resolution, ancient strata collapsed into the blur. Storage is **Θ(T) if every level of the pyramid is kept** (the fan-in gives a geometric series $T \cdot (1 + 1/N + 1/N^2 + \ldots)$), and becomes **sublinear only under bounded retention per level** — a configuration this section does not pin.

Pressure-driven (normative). The descent rate and the pyramid's level transitions are driven by **pressure** (disk pressure, age, or an explicit force), never a fixed schedule. Pressure sources, slowest→fastest: natural aging < scheduled retirement < capacity (disk) eviction < **revocation (immediate forced descent below the floor)**. Capacity pressure is **arbitrated per owner** by the CC 6.1.5.2 storage-contention axis — cache and over-budget corpus descend before pinned corpus — but revocation still overrides pin state (CC 6.1.5 N5).

Forgetting and erasure converge (CC 6.1.5 N5). The N5 erasure guarantee is exactly "not individually recoverable at or below the noise floor." A sufficiently-aggregated composite **may be** already-erased by degradation — **but only when no single source dominates it**. The naive "a picture of a thousand pictures contains `< 1/N` of any source" holds for **independent, non-dominated** sources and **fails for outliers**: a unique color, a rare position, or a near-duplicate cluster can dominate a composite, so the blur *is* that subject. The already-erased shortcut is therefore **conditional on a source-diversity gate**: the id-based `member_commitment` (CC 6.1.2.1.1) counts *which* members and never *dominance* or *content* multiplicity, so `AggregationMetaV1` **v3** carries the two signed members that make the gate checkable on the wire — `mass_commitment` (mass dominance, provable from held evidence) and `max_source_multiplicity` (content-similarity multiplicity, the R9 residual a mass-only surface cannot see) — carried in the CC 6.1.2.1 preimage and enforced at admission by the CC 6.1.2.1.2 gates. Where the gate fails, revocation's purge obligation applies in full (CC 8.3.1 R9). No purge is needed **only** where the shortcut's diversity precondition holds. Revocation simply *forces* an item below

the floor **now**, and MUST purge only the retained tiers where it is **still individually recoverable** (the high-fidelity upper layers). It need not — and MUST NOT be required to — destroy the collective gist. Capacity-eviction reaches the identical end-state gradually. Same destination; revocation just gets there first.

Infrastructure is self-sufficient for memory (sharpens CC 3.4.7.3). Both degradation operators are **mechanical** (symbol arithmetic + resampling) — they require **no reasoning and no agency**, so a pure fabric node performs the entire retention function. A brain MAY enrich a degraded tier with a richer semantic gist, but is **never required**: infrastructure remembers without agency. The mechanism is mechanical degradation; the brain is optional enrichment. The degradation operators act on the witnessed corpus a fold holds in the clear; the blob layer of CC 6.1.5 holds sealed chunks and never opens them — two objects, and the fabric node's retention function is the first.

Disposition mapping. The CC 6.1.5 `EjectionVerdict` values are points on this one axis: `Keep` = above-floor, no pressure; `EjectToTier` = a downward step (still recoverable at lower fidelity — by **dropping whole layers**, never by keeping part of a layer's chunks); aggregation = N→1 downward step; `EjectHardDelete` = forced descent below the floor + purge-still-recoverable-tiers. They are not distinct mechanisms — they are stops on the single pressure-driven descent.

6.1.2.1 `AggregationMetaV1`: the aggregation-tier wire contract (normative)

The metadata that tags one tier of the CC 6.1.2 memory pyramid: which content, at what aggregation tier, over which source members, by which mechanical operator. This shape is **CEG-canonical** — the reference implementations store `aggregation_meta` **opaque** (the wire-churn firewall), so this section **defines** the byte layout and implementations conform to it.

`AggregationMetaV1` is a **substrate wire shape, NOT a CC 2.1 attestation** — no 1+4 change. Its signing preimage uses the CC 6.1.3 binary discipline (length-prefixed, big-endian, domain-separated — **NOT** CC 2.6.1 JCS). Preimage byte order (normative):

```
preimage = b"AGG-META-v1\0\0\0\0\0" // 16-byte domain separator (exact; unchanged at v3)
 || u32_be(version = 3) // current; see Version below
 || lp(content_id) // the root content this pyramid is for
 || lp(corpus_kind) // "trace" | "blob" | "av_chunk" | ...
 || u32_be(tier) // 0 = source granularity; higher = more aggregated
 || lp(aggregation_algorithm_id) // opaque codec id, e.g. "layered-gist-v1"
 || u32_be(source_count) // N members aggregated into this tier (descent fan-in)
 || member_commitment[32] // CC 6.1.2.1.1 Merkle root over the source member ids
 || lp(noise_floor_descriptor) // what survives below the floor (codec-specific, canonical)
 || u32_be(n_eff) // v2+: signed effective source count (CC 6.1.2.1.2)
 || u32_be(max_source_multiplicity) // v3: largest content-similarity cluster size (R9-b)
 || mass_commitment[32] // v3: Merkle root over the per-member masses (R9-a)
// lp(x) = u32_be(byte_len(utf8(x))) || utf8(x) // length-prefixed UTF-8
```

The version members are **appended in version order** — `u32_be(n_eff)` at v2, then `u32_be(max_source_mult` ‖ `mass_commitment[32]` at v3 — so the fields are appended in version order; the complete preimages differ at the `version` field and the domain separator is unchanged; discrimination is carried by the `version` field alone. This byte order is golden-vectored (`aggregation_meta/canonical_bytes_v3`) and reproduced cross-impl; re-ordering it is a vector-invalidating wire change.

Version (normative). `version = 3` is the only current value, with **no deprecation window**: a verifier MUST reject a tier at `version < 3` as inadmissible. The CC 6.1.2.1.2 R9 multiplicity gate requires `version` $\geq$ `3` (only v3 carries `max_source_multiplicity`) and the mass-dominance gate over the signed `n_eff` requires `version` $\geq$ `2`; both fail **closed** on a lower version.

`content_id`, byte-valued ids, `member_commitment` and `mass_commitment` are lowercase-hex per CC 2.6.3 where rendered as strings; both commitments on the wire are the raw 32 bytes. **Bound-hybrid signature** (the CC 6.1.3 rule): `Ed25519(preimage) + ML-DSA-65(preimage ‖ ed25519_sig)`; a verifier MUST reject a tier lacking a valid ML-DSA-65 half **at ingest and before persistence** (the CC 5.3.2.4.3.1 store-path rule applies — `AggregationMetaV1` is federation-tier).

6.1.2.1.1 `member_commitment` (descent integrity)

`member_commitment` is the Merkle root over the **source member ids aggregated into this tier**, computed by the **CC 6.1.1 wholeness witness Merkle construction** (same `leaf = SHA-256(utf8(member_id))`, **lexicographic** leaf order, `node = SHA-256(left ‖ right)`, odd-node duplication, and empty-set sentinel) — reused deliberately so the federation carries **one** aggregation/witness Merkle scheme, not a third. It uses the CC 6.1.1 scheme with **no** RFC-6962 prefix; safe here because `member_commitment` is verified by **full source-id-list recomputation**, never partial inclusion proofs, so the odd-node-duplication malleability is moot given the deduplicated-leaf rule of CC 6.1.1 (a repeated `member_id` is refused; the source-id list is sorted AND unique). `member_commitment` lets any verifier confirm a tier was aggregated from exactly the claimed sources without holding the sources.

6.1.2.1.2 `n_eff`, the dominance gate, and the R9 multiplicity gate (normative)

The two admission gates that make the CC 6.1.2 source-diversity precondition checkable on the wire. `member_commitment` (CC 6.1.2.1.1) commits to *which* members a tier folds — never to *how much of the composite each supplies* (mass), nor to *how many members are the same content under different ids* (multiplicity). The v2 and v3 signed members close those two blindnesses in that order.

`n_eff` and the mass-dominance gate (v2+, normative). The aggregator computes, at fold time, the **effective source count** of the tier from its per-member content masses `m_i` — the Kish / inverse-Simpson participation ratio, taken to the integer floor over integer masses:

```
n_eff = floor( (sum m_i)^2 / sum m_i^2 )   over integer masses m_i > 0,
                                           m_i = mass_to_fixed(fraction_i)
```

The masses are the fixed-point integers `mass_commitment` commits (`mass_to_fixed`, below), so aggregator and verifier compute over the same integers; the ratio is scale-free, and under floor no tie is possible.

A balanced fold of N equal-mass sources has `n_eff = N`; a fold where one source supplies ~90% of the composite collapses toward `n_eff → 1`. An empty or all-zero-mass fold has `n_eff = 0` (fail-closed: no positive mass, no effective sources). `n_eff` enters the signing preimage at `version ≥ 2` (CC 6.1.2.1), so the value a verifier reads is the value the aggregator signed — and since v3 a *lying* `n_eff` is **checkable against `mass_commitment` from held evidence exactly** — a verifier recomputes the floor over the committed integer masses, and any difference is the lie. (The two "Kish" objects in this section are distinct: this `n_eff` is a mass-weighted participation ratio over sources; the `k_eff` of the effective-count paragraph at the end of this section is the equicorrelated variance-of-mean count.) A tier passes the mass-dominance gate iff `version ≥ 2`, `source_count > 0`, and

```
n_eff >= min_ratio * source_count      with pinned min_ratio = 0.5
```

`min_ratio = 0.5` is **CEG-pinned** — a normative constant, not an implementation-tunable knob (two conformant substrates MUST NOT disagree on which folds are admissible). The gate MUST run **after** signature verification (which authenticates the `n_eff` it reads) and **before persistence** at aggregated-tier admission; a failing tier is rejected with the stable token `aggregation_meta_dominated`.

R9 (content-multiplicity gate) — CEG-pinned constants (normative, v3). `max_source_multiplicity` and `mass_commitment` are covered by the bound-hybrid signature, so a divergence in how they are *computed* is not cosmetic: it is a fork in admissibility, with valid signatures on both sides. They are therefore **pinned normative constants, not implementation-tunable knobs** — the same discipline as `min_ratio` above and the pinned `(R, ε)` default in CC 6.1.2. A conformant implementation MUST use:

- **similarity metric** — `1 - normalized L1 distance` over the resampled payloads, evaluated **entirely in integer / fixed-point** arithmetic. An IEEE-754 decision path MUST NOT be used: it can fork admissibility between two honest implementations on byte-identical inputs. (Cosine is excluded by construction — byte vectors are all-positive, so even independent members score ≈ 0.75.)
- **threshold** — `0.950`, carried as the integer `950` in milli-units. Calibration (measured): byte-identical $\to$ `1.000`; near-duplicate $\to \geq$ `0.99`; independent high-entropy $\to \approx$ `0.667` (mean $|\Delta|$ of uniform bytes $\approx 85/255$).
- **clustering** — the largest **connected component** of the similarity graph. This is a deliberate conservative superset of the max-clique reading (max-clique is NP-hard); it can only **over**-estimate multiplicity, which only ever **tightens** the gate — the fail-safe direction for a privacy gate.
- **resample basis** — the **same** nearest-neighbour resample the fold applies; similarity MUST be measured on exactly the content the fold collapsed.
- **`n_min`** — substrate-side and corpus-kind-pinned (default `2`). It MUST NOT be producer-settable: a producer must not be able to widen its own admissibility.
- **`mass_commitment`** — the CC 6.1.1 Merkle construction over per-member leaves `leaf_bytes = utf8(member_id) ‖ u64_be(mass_to_fixed(mass))`, where `mass_to_fixed(m) = round(m × 1e6)` as a `u64` and any `m ≤ 0` commits as `0`. As everywhere in the CC 6.1.1 scheme, leaves are sorted lexicographically over their **raw leaf bytes** before hashing (`leaf = SHA-256(leaf_bytes)`). It makes a lying multiplicity provable from held evidence, and a lying `n_eff` checkable exactly, since `n_eff` is the floor over these committed integers.
- **gate** — a fold is multiplicity-admissible iff `max_source_multiplicity · n_min ≤ source_count`.

The R9 negative these constants make derivable from this document: 900 near-duplicate contents folded under 900 distinct ids at equal mass are perfectly mass-honest (`n_eff == N`, so the dominance gate admits them) and are rejected only by the multiplicity gate — which is why a mass-only surface was insufficient and why v3 exists. This closes the wire-visibility half of the R9 residual; it does **not** close the adversary-model limb of the CC 6.1.2 below-floor MUST, which remains unverifiable-pending-instrument.

The effective count (normative). The effective number of independent sources behind `k` distinct sources whose average pairwise correlation is $\bar{\rho}$ is the Kish effective count:

```
k_eff = k / (1 + rho_bar * (k - 1))
```

An implementation clamps $\bar{\rho}$ to `[0, 1]` before applying it: positive semidefiniteness admits negative $\bar{\rho}$, and the identity then yields `k_eff > k`, but asserted anti-correlation is exactly what an adversary fabricates to mint extra credit, so the clamp credits at most `k`. At $\bar{\rho}$ `= 0`, `k_eff = k`; as the sources correlate, `k_eff` falls; at $\bar{\rho} \to$ `1`, `k_eff` $\to$ `1` and never below. **Fully correlated sources count as one** — this is the correlated-source discount: a clique of any size whose members move as one contributes at most one independent source, and no number of copies of one voice buys more. For $\bar{\rho}$ `> 0` the count has a ceiling, `k_eff` $\leq$ `1/`$\bar{\rho}$, that scale cannot lift. $\bar{\rho}$ is read from the pairwise correlation an implementation measures over its sources (co-signing frequency, shared steward lineage, temporal clustering — the CC 3.1.8.4 F-3 matrix), over attested event time and never over polling-window boundaries. Because $\bar{\rho}$ is an average, a tight clique embedded among independent sources is under-penalised: a computed `k_eff` can **overstate** effective diversity and never understates it. A threshold that **denies** on low `k_eff` is therefore safe, while a threshold that **grants** on high `k_eff` is the exposed direction and MUST NOT be the sole gate for any capability. The mass-weighted `n_eff` above is the same Kish construction over per-member weights; the two are computed separately and are not substituted for one another. Mechanised: `lean:coherence-ratchet:Core.BaseIdentity` (the definition, its bounds, its monotone decrease in $\bar{\rho}$ and the ceiling) and `Core.SignalSourceDiscount.clique_neutralization` (a fully correlated clique counts as one). This document states **no operating band** for $\bar{\rho}$, for `k_eff` or for any ratio derived from them, and an implementation MUST NOT gate on one.

6.1.2.2 Descent rule (normative)

`descend(content_id, corpus_kind, tier)` $\to$ `[member_id]` returns the **ordered** source members aggregated into the tier-`tier` composite — the tier-`(tier-1)` members one level down the pyramid. It MUST be a **pure, deterministic** function: two impls return the **byte-equal ordered list** for byte-equal inputs. The order is the **lexicographic member-id order** `member_commitment` (CC 6.1.2.1.1) committed to — so a returned list re-derives the parent's `member_commitment` byte-for-byte (the descent-integrity check).

Ordinary descent never terminates at zero (the retained floor). For witnessed content under ordinary retirement, below tier 0 (source granularity) the content's **collective gist persists as the lowest retained tier** — a composite whose members are no longer *individually* recoverable (it is **below the noise floor**, CC 6.1.2) but whose blur survives. `descend` past the noise floor yields the blur, never an empty/destroyed object. The function is **pressure-independent** (pure navigation); **pressure drives which tiers are *retained*** (CC 6.1.2), not the descent computation. Ascending (aggregation, operator 2) is the N$\to$1 inverse with fan-in `source_count`.

Prohibited-content carve-out. *(normative; the retained floor is for ordinary retirement only)* Content under a CC 4.5.3 immediate-removal `legal_basis` (`NcmecCsam`, `PerceptualHashCsam`, `GifctCip`, `CourtOrder`) terminates at **zero**: the payload is destroyed at every tier, no blur survives, and only the **content hash** is retained (for the legal retention window) so other nodes may refuse the same payload on arrival without ever holding it. An implementer reading this Part alone MUST NOT build blur-forever for prohibited content — the two rules cross-reference each other by design. Stated honestly (the delivery limit): a purge attestation rides the pull-only replication plane, so *"we emitted a purge" is not "the bytes are gone"* — authority is per-mesh and delivery is not guaranteed. What the federation guarantees is that a conformant holder **destroys on receipt** of the purge (MUST, not SHOULD, for these bases), that the hash lets any conformant node refuse re-admission indefinitely, and that a holder that is offline, forked,

de-admitted, or joins later is outside this mesh's enforcement reach and is said to be — the obligation on reachable holders is total, and the limit on unreachable ones is stated rather than implied away.

6.1.2.3 `EjectionVerdict`: the tier-aware retirement surface (normative)

The single verdict surface a verifier exposes and a substrate consumes to gate one step of the CC 6.1.2 descent. CEG pins it as the canonical superset of the rarity-only `RetentionDecision`:

```
EjectionVerdict::= Keep // above the floor, no pressure step
  | EjectToTier // one downward step: still recoverable, lower fidelity
    // (intra-object layer-drop OR N->1 aggregation -- never a
    // partial layer; a layer is held whole or not at all)
  | EjectAggregatedTierOnly { tier }
    // shed exactly one pyramid stratum -- the tier-`tier`
    // composite -- leaving finer AND coarser tiers intact
  | EjectHardDelete // forced descent below the floor + purge still-recoverable tiers
```

Mapping (normative): `RetentionDecision{RetainRare|RetainNonRare|EvictEligible}` is the rarity sub-decision *within* `EjectToTier`/`Keep`; `EvictEligible` + capacity pressure → `EjectToTier`; **cache or over-budget corpus (CC 6.1.5.2 B5) → `EjectToTier` ahead of pinned content**; `EvictEligible` + a `withdraws`/`consent:state:revoked` (CC 6.1.5 N5) → `EjectHardDelete` **regardless of pin state** (the fastest descent, never tier-shed — CC 6.1.2). **`EjectAggregatedTierOnly { tier }`** is the tier-granular form of `EjectToTier`: it sheds a single intermediate stratum of the CC 6.1.2.1 pyramid (the tier-`tier` `AggregationMetaV1` composite) under targeted pressure, leaving both finer and coarser tiers — composing with the hard-delete trait (a `tier` below the noise floor is unreachable, so this never resurrects erased content). A pure fabric node MAY compute `EjectToTier` / `EjectAggregatedTierOnly` mechanically; `EjectHardDelete` MUST purge per CC 6.1.5 N5.

Conformance. CC 6.1.2.1–.3 are gated at CC 6.1.4 by the v3 vector family. The CC 6.1.3 binary-length-prefixed discipline makes wire-identity reproducible from the text alone. `member_commitment` reuses the CC 6.1.1 wholeness witness Merkle **verbatim** (same `WW-v1-empty` sentinel) — the federation runs **one** Merkle scheme across CC 6.1.1 (witness leaves) and CC 6.1.2 (member commitments), no schema fork.

6.1.3 Canonicalization boundary + the 1+4 line (normative)

This is the seam that protects the frozen attestation envelope: every CC 6.1 object is *substrate framing*, never an attestation, so it never touches the 1+4 surface or its canonicalization.

- **The frozen 1+4 attestation envelope (CC 2.1) is untouched.** Every CC 6.1 object is **transport/substrate framing** — it never instantiates a CC 2.1 Contribution, never adds an `attestation_type`, never enters CC 2.6.1 JCS canonicalization. The realtime A/V chunk wire (CC 5.3.3.2.3) is the same category.
- **CC 6.1 uses a binary, length-prefixed, big-endian, domain-separated signing preimage — NOT CC 2.6.1 JCS.** These are verify-to-verify transport primitives that never cross the four-impl boundary as JSON (the same boundary CC 5.3.2.4.2 drew for Verify's `signing_bytes` framing). An implementer MUST NOT apply JCS to a CC 6.1 object or its signatures will not verify cross-impl. Each object's domain separator (`b"CIRISALMCAPv2\0\0"`, `b"ciris-edge/holding-claim/v1"`, `b"ciris-edge/compress-request/v1"`,

`b"CIRIS-CLAIM-v1\0\0"`, the wholeness witness `b"WW-v1-empty"` empty-sentinel, etc.) is pinned by its subsection.

- **PQC-mandatory (CC 5.3.2.4.3.1) binds every CC 6.1 signed object.** Each carries the bound hybrid pair (Ed25519 over the canonical preimage; ML-DSA-65 over `preimage` ‖ `ed25519_sig`); a verifier MUST reject a CC 6.1 object lacking a valid ML-DSA-65 half **at ingest and before persistence** (no store-then-quarantine — the store-path rule). Verification happens **at the gate**: an admission/verdict function MUST verify signatures itself and MUST NOT trust an in-band `verified` flag (such a flag MUST be non-wire).
- **What is wire vs internal (the CC 1.13.4 line).** Cross-impl-observable bytes (signed preimages, content-addressed hashes, and the deterministic topology output) are **PIN-NORMATIVE**. Local heuristics whose output no other peer reproduces are **edge-internal** and MUST NOT be over-pinned: specifically **ALM parent-selection** (over per-peer RTT/reachability) and `retention_priority` (never on the wire) are edge-internal; **rarity scoring** is a **recommendation**, not a MUST.

6.1.4 Conformance: the freeze gate

The byte-exact signed preimages and the `compute_alm_topology` output are pinned against the reference impl, and **conformance vectors generated from it are the freeze gate**: input → expected bytes for `SealedAvChunk` + the two AV nonces, `SignedRelayCapacity`, ALM topology (input snapshot → expected tree hash, incl. permutation invariance), the holding claim and compress request of CC 6.1.5 + `retention_priority`, `StorageBudgetV1` (per-`cohort_scope` allotment) + `CorpusWantV1` (the CC 6.1.5.2 want/have negotiation), and `WholenessWitness` canonical bytes + Merkle root (incl. the empty sentinel + odd-node duplication). The **v3 vector family** — the `AggregationMetaV1` preimage and signature, `member_commitment`, `mass_commitment`, the `descend` ordered output, the three calibration points as the executable similarity vector and the 900-near-duplicate fold as the R9 negative — **is the conformance gate for `AggregationMetaV1`**, and the register records who has reproduced each family. Beyond wire conformance, **no operational benchmarking layer for the CC 6.1 mechanisms is specified anywhere in this document** — CC 8.8.10 Annex J is the HE-300 ethics harness and instruments none of them. That gap is stated, not filled.

6.1.5 Durable storage: per-item replication and swarm rarity

Durability is a **per-item choice the owner makes on publish**. An item marked **durable** — at `family` or `community` scope — carries a **target holder count**; the blob swarm replicates the item's sealed chunks to that count across the cohort's **live** nodes, **rarest chunk first**; repair runs when the inventory (CC 6.1) shows the item's holders falling toward its floor; recovery is *pull the chunks from any holder and decrypt with the recovered cohort key* (CC 5.1); an item that is un-marked, or withdrawn, **descends** — fewer holders, then none (CC 6.1.2.2). Where an item is layered (CC 5.3.3.2.4 `ChunkLayer`) each layer is its own chunk set, so a layer can be shed whole. Durability policy is the owner's, per item, bounded by the cohort, and never the substrate's invention. Nodes and relays never encode or decode stored content: a chunk is sealed once (CC 5.4.2) and is held and forwarded as sealed bytes. This is the durability layer M-1's "shared memory" rests on — but it is held subordinate to consent, so durability never overrides a withdrawal.

The room at row level. Every anti-entropy round carries what a node holds and lacks, per plane; the noise-floor aggregation producer (CC 6.1.2.1) runs over those rounds, and the converged

view is a tap over them. The rest of this section states the rule holding claims, rarity, repair and descent are built to.

- **Holder directory (no duplication)** — a **holding claim** ("node X holds chunks S of item C") is the chunk-level companion of the whole-blob `custody:ack:v1` report (CC 3.1.3.3), placed at the content's scope as it is and reusing its liveness window + `ContentMiss` feedback. It MUST NOT create a second who-holds-what directory. No holding claim leaves a bounded cohort: for `cohort_scope: self | family | community | affiliations` content a holding claim is **scoped to the content's own cohort** — emitted to, admitted by and relayed among that cohort's own nodes only, never federated — so holding claims cannot leak the existence of structurally-invisible blobs to anyone outside (CC 6.1.5.3).
- **N5 (retention respects revocation — fail-secure)** — retention MUST NOT keep alive, **above the CC 6.1.2 noise floor**, content whose consent is withdrawn (CC 2.4.1.1) or revoked. A withdrawn `content_id` is descent-eligible **regardless of rarity**; an active `withdraws` / `consent:state:revoked` overrides the max-rarity "keep" signal and forces immediate descent below the noise floor (the fastest form of the one retirement operation — see CC 6.1.2); unknown consent state defaults to *not retained as rare*. The CC 4.4.3.5.2 deletion-SLA + CC 4.4.3.5.1 decay stages take precedence over swarm coverage at all times. **Revocation does not require destroying the collective gist** — only that the item be **not individually recoverable** at any retained tier (CC 6.1.2); it MUST purge any tier where it still is.
- **N6 (possession-bound claims)** — a holding claim counted toward an item's holders or toward rarity MUST be possession-challengeable (a holder answers a chunk request, or the claim carries a proof-of-possession). Unverified holding claims MUST NOT lower another node's retention priority — otherwise rarity is a forgeable force-evict channel. Where no challenger runs, the **weight rule** applies: a claim whose possession is verified counts 2, a claim that is only signed counts 1, and an unverified claim counts 0.
- **N7 (chunk integrity)** — a holder and a recovering node MUST verify each chunk against the item's signed per-chunk SHA-256 (a swarm-sourced chunk cannot poison a recovery).
- **SR-2 / SR-3 (anonymous + recovery scope)** — anonymous-tier content is **exempt from swarm-mandatory retention** (governed by LRU-only): no holding claim, no compress request, no rarest-first biasing. The "recovers from any holder" property is a property of the **witnessed, trust-anchored** corpus **only** — it does not extend to anonymous content, which the substrate MUST be able to let truly disappear.
- **PIN line** — the holding-claim and compress-request signed preimages = **PIN-NORMATIVE** (with chunk ids sorted ascending before signing); `compute_rarity_score` = **PIN-AS-RECOMMENDATION**; `retention_priority` = **edge-internal** (never on the wire).

6.1.5.1 Replication-target policy: normative floor and recommended defaults

Per-item policy (normative). The target holder count is the **owner's choice per item**, set on publish and bounded by the cohort: a target MUST NOT exceed the item's audience (CC 6.1.5.3), and the substrate MUST NOT raise a target an owner did not set. Two floors sit under every target, each counted in **holders per item**: a **minimum holders** floor, at which repair runs, and a **minimum viable** floor, below which the item is no longer served and descends to EnvelopeOnly (CC 6.1.2) — the signed envelope survives, never zero. A CEWP-1.0 conformant node:

- MUST set minimum viable `>= 1` — the EnvelopeOnly tier is locked at the substrate, never an unbounded floor of 0.
- MUST be able to hold items at **any** target a cohort it joins publishes — a node MUST NOT hard-code one target and refuse others.
- MUST read the gap to target from the inventory (CC 6.1) and nothing else: holders per item are counted from live, possession-weighted holding claims (N6), and an absent or lapsed claim is *unknown*, never a deficit of one.

Defaults are the steward's, through the plane. An owner who marks an item durable and names no count takes the cohort's defaults. Those defaults — **repair count**, **minimum viable**, **target holders**, **minimum holders**, each over holders per item — are the two redundancy keys of the mesh-config plane, target holders and minimum holders (CC 4.2.1); this document pins no number for them, and the reference's shipped values are defaults an owner's per-item choice overrides.

The mesh-config loop (normative). Measurement changes a node's behaviour through exactly one path: the mesh-config plane — steward-authored, time-limited, and able only to **relieve**: hold fewer copies, raise a floor, pause a plane, slow a cadence. A `mesh_config` key MUST NOT make a node hold or send more than its owner consented to (relieve-never-expand, most-restrictive-across-roots — CC 4.2.1 rule (1)), and a key naming no consumer processor is refused at admission (rule (2)). The keys are: the four live ones — the anti-entropy round cadence, the page limit, the trace-replication pause and the A/V pause; the two redundancy keys above, whose consumer arrives with the durability tier (until it does, rule (2) refuses them); and the shaping and onion keys — `shaping.rho`, `shaping.window_secs`, `shaping.node_cap_bytes_per_s` (CC 5.4.3) and `onion.cap_factor`, `onion.mix_mean_delay_ms` (CC 5.4.6) — each relieve-never-expand. The loop is: nodes publish what they hold and lack, and send their census returns (CC 6.1.9); the folds produce the inventory and the census; a steward reads them and signs an expiring relief; nodes consume the relief. A node MAY always do less of its own optional work without any row (CC 4.2.1.4); it never does more on the census's account.

Reliability (informative). With a holder a whole copy, an item is recoverable while at least one holder is live: for independent holders at liveness `q`, `1 - (1 - q)^h` — 0.999 at `h = 3`, `q = 0.90`. The devices of one family are not independent (one house, one outage), and the correlation-discount discipline of this Part applies before any such figure is quoted. The measured figures for the rateless codec describe the realtime A/V path (CC 5.3.3.2) and are not a storage claim.

No wire change. This policy pins a floor and a plane over the owner's per-item target; it introduces no new attestation shape and no 1+4 change.

6.1.5.2 Owner storage budget and pin-on-consent (normative)

Replication is otherwise bounded only by **wire type** (CC 5.3.2.3), **membership** (`cohort_scope`), and **consent** (`consent:replication`, CC 3.3.7) — nothing bounds **resource contention on an owned node**. The eviction ladder (CC 6.1.2) is purely *reactive*: it fires only after content has already landed. This section adds the missing axis — an owner bounds what replicates onto their own node **before it arrives**, and pins durable content by consent. Because the substrate MUST NOT invent replication policy (CC 5.3.2.3), this bound is a **declared CEG rule**, not implementation-defined per node. It is a pinning model under a size cap: durable content is pinned; everything else is cache, collected under the cap.

- **B1 (pin classes).** Identity, consent, and config records are **always pinned** — small, identity-critical, durable, and **exempt from the byte budget**. Corpus is **pin-on-consent, cache-otherwise**.
- **B2 (pin-on-consent).** A corpus `content_id` is **pinned** (durable) iff **both** hold: (i) an active `consent:replication` grant (CC 3.3.7) authorizes the corpus `subject_kind` covering it — the grant **is** the pin authorization (no separate pin dimension); a `consent:replication` grant names corpus `subject_kind`s in its authorized-class list, extending its attestation-prefix grammar to the corpus track of CC 5.3.2.3; **and** (ii) the owner **elects** to spend budget on that class by listing it in the `pinned_class` set of its `StorageBudgetV1` (B3). Consent *authorizes*, the owner *elects* — a pin requires **both**. Corpus received without a satisfied pin is **cache**: eviction-eligible, and it descends **before** any pinned content under pressure (CC 6.1.2.3). This is the owner's "bound what lands on me" control — unconsented corpus never becomes durable on the owner's node.
- **B3 (owner budget — per `cohort_scope`).** An owner declares a `StorageBudgetV1` (below) allotting, **per `cohort_scope`**, a `budget_bytes` ceiling and a `pin_reserve_bytes` floor (MUST be $\leq$ `budget_bytes`) held for pinned corpus. Pinned corpus in a scope MUST fit its `budget_bytes`; content that would exceed it is refused at admission (via the B4 want/have handshake) or, if already held, is first to descend under contention (B5). A `StorageBudgetV1` **inherits the CC 5.2 `self | family` suppression**: `self` and `family` scope entries MUST NOT appear in a signed / federated budget — those budgets are enforced **locally only**, else the budget would leak the existence of structurally-invisible content (the same hazard the CC 6.1.5 holding-claim suppression closes). Budgets are **supersedable** by monotonic `revision` — the **anti-rollback** aspect of CC 5.3.2.3 (a single-owner self-declaration, not a multi-signer quorum): a higher `revision` from the same `node_id` supersedes; a lower one MUST be rejected.
- **B4 (want/have + size cap).** Large corpus is **wanted-then-pulled**, never unsolicited-pushed: a peer advertises a `want` for content **within its remaining scope budget** and a `size_cap`; a producer MUST NOT push a corpus object exceeding the receiver's advertised `size_cap`. This is at once a consent control and a bandwidth control on constrained transports. Content is content-addressed (CID-style): a pin is a stable reference and dedup is free — a `content_id` pinned by several scopes occupies its bytes **once** and is retained while **any** scope pins it.
- **B5 (contention arbitration).** When a scope is over budget under disk pressure, descent order (CC 6.1.2.3) is deterministic: **cache before pinned**; within pinned, **lowest rarity (CC 6.1.5) first**; ties broken by **oldest `revision`**. A `StorageBudgetV1` is **consumption-challengeable** the way CC 6.1.5 N6 makes holding claims challengeable — an owner's declared consumption MUST be reconcilable against the pinned `content_id`s it actually holds, so a forged budget cannot become a force-evict channel against a competitor's content. Per-scope **consumption accounting is edge-internal** (recomputed from held content), never trusted from the wire.
- **B6 (consent supremacy preserved).** Pinning **never** defeats consent: an active `withdraws` / `consent:state:revoked` still forces immediate descent below the noise floor regardless of pin state (CC 6.1.5 N5). A pin holds content **above** the floor against **capacity** pressure only — never against **revocation**. This section is the positive inverse of N5, and is bounded by it.

`StorageBudgetV1` — the owner's per-`cohort_scope` allotment (normative wire shape). A substrate-framing object, **not** a CC 2.1 attestation — no 1+4 change. Its signing preimage

uses the CC 6.1.3 binary discipline (length-prefixed, big-endian, domain-separated — **NOT** CC 2.6.1 JCS):

```
preimage = b"CIRIS-STG-BUDGET"                 // 16-byte domain separator (exact)
 || u32_be(version = 1)
 || lp(node_id)                                 // the owner node this budget binds
 || lp(epoch_id)                                // epoch keying (CC 5.1)
 || u64_be(revision)                            // monotonic; higher supersedes (anti-rollback)
 || u32_be(scope_count)
 || scope_count x (                             // one entry per cohort_scope; NEVER self/family (B3)
       lp(cohort_scope)                        //   "community" | "affiliations" | "species" | ...
     || u64_be(budget_bytes)                    //   total ceiling for this scope
     || u64_be(pin_reserve_bytes) )             //   floor reserved for pinned corpus (MUST be <=
    budget_bytes)
 || u32_be(pinned_class_count)
 || pinned_class_count x lp(subject_kind)       // corpus classes the owner elects to pin (B2-ii)
// lp(x) = u32_be(byte_len(utf8(x))) || utf8(x)
// cohort_scope entries AND each subject_kind list: sorted lexicographically over UTF-8 bytes,
    deduplicated
```

Bound-hybrid signature (the CC 6.1.3 rule): `Ed25519(preimage)` + `ML-DSA-65(preimage || ed25519_sig)`; a verifier MUST reject a `StorageBudgetV1` lacking a valid ML-DSA-65 half **at ingest and before persistence** (the CC 5.3.2.4.3.1 store-path rule — `StorageBudgetV1` is federation-tier). A higher `revision` from the same `node_id` supersedes; a lower one MUST be rejected (anti-rollback).

Validation (normative). A verifier MUST **reject** a `StorageBudgetV1` if any `pin_reserve_bytes > budget_bytes`; if the `cohort_scope` entries or any `subject_kind` list are not lexicographically sorted and deduplicated; or if a `self` / `family` scope entry is present (B3 suppression). Two `StorageBudgetV1` from the same `(node_id, revision)` with differing content are **equivocation**: a peer MUST retain and surface both as a `hard_case:*` (CC 3.4) and MUST NOT silently pick one — mirroring the CC 6.1.1 N4 rule.

`CorpusWantV1` — the B4 want/have advertisement (normative wire shape). A peer advertises exactly what corpus it will accept and its per-object ceiling; a producer pulls only against it. Same CC 6.1.3 binary discipline:

```
preimage = b"CIRIS-WANT-HAVE\0"                // 16-byte domain separator (exact; one trailing NUL)
 || u32_be(version = 1)
 || lp(node_id)                                 // the advertising peer
 || lp(epoch_id)                                // epoch keying (CC 5.1)
 || lp(cohort_scope)                            // the scope this want draws budget from (NEVER self/
    family)
 || u64_be(size_cap_bytes)                      // max single-object size this peer will accept
 || u64_be(remaining_budget_bytes)              // advertised headroom in the scope
 || u32_be(want_count)
 || want_count x lp(content_id)                 // content-addressed ids wanted; lexicographic,
    deduplicated
// lp(x) = u32_be(byte_len(utf8(x))) || utf8(x)
```

Bound-hybrid signed and verified as above; **PIN-NORMATIVE**. A producer MUST NOT push a corpus object whose size exceeds the advertised `size_cap_bytes`, nor any object whose `content_id` is absent from an active `CorpusWantV1` from the receiver (**wanted-then-pulled, never unsolicited-pushed**). A `CorpusWantV1` MUST NOT name a `self` / `family` `cohort_scope`.

PIN line. `StorageBudgetV1` and `CorpusWantV1` signed preimages = **PIN-NORMATIVE** (all `cohort_scope`, `subject_kind`, and `content_id` lists sorted lexicographically over UTF-8 bytes and deduplicated before signing); per-scope **consumption accounting = edge-internal** (recomputed from held content per B5, never on the wire).

Scope. This section supplies the **technical** storage bound (quota + budget + pin); the **economic** *who-pays* dimension is deliberately off-wire (CC 3.3.9 billing) and is not part of these shapes.

6.1.5.3 Durability at every tier (normative)

The direction. The per-item target of CC 6.1.5.1 and the chunk replication of this section are **available at every tier**, and **anti-entropy is the standard carrier at every tier**. CC 5.4.2 seals self/family records chunk by chunk; the rules below give those tiers holding claims, rarest-first, repair and a measurable target.

Anti-entropy is standard at every tier (normative). Every row and every blob a node is audience for reaches it by **pull anti-entropy** on the one replication carrier — never by push, and never waiting on a user's action. At `self` and `family` scope the rounds run among the **cohort's own nodes** (a person's occurrences and the nodes they own; the family roster's devices), over the derived plane or a direct link and, where no direct path exists, the CC 5.4.6 identity-link fallback. At `community` / `affiliations` they run over the cohort roster, and at federation scope over the commons. No tier is exempt except the opt-in anonymous tier, which SR-2 above lets truly disappear.

Holding claims at every tier, bounded to the cohort (normative). A holding claim (chunk level, CC 6.1.5) and a `custody:ack:v1` report (whole blob, CC 3.1.3.3) exist at every tier, placed at the content's own scope: within a bounded cohort (`self`, `family`, `community`, `affiliations`) both are replicated only among its members' nodes and never reach a non-member, so an outsider learns nothing it did not learn before; at the Commons both are federation-scoped and are the holder directory (CC 3.1.3.3). N6 (possession-challengeable claims) applies within the cohort exactly as outside it.

The audience is claimed nodes, under their owner's choice (normative). Every CIRIS node is **claimed** before it operates on the mesh in normal operation — it carries a live owner-binding to a person (CC 3.2) — and a node's audience is **its owner's cohorts**: a claimed node is in a cohort's audience only because its owner is a member of that cohort, and an unclaimed node is in no cohort's audience. A home node is therefore not a special case; it is a claimed node of its owner, and through that claim it holds what its owner's cohorts hold. The owner then **chooses, per node, which of their cohorts it serves**: an allow or deny list over the cohorts the owner belongs to ("don't replicate that community's content onto my work laptop"). A node a cohort is denied on is **not in that cohort's audience**: it is not counted toward the target, is not a deficit, receives no keys for the cohort and pulls nothing for it — enforced at both ends, by the sender's recipient predicate and by the receiver's hold decision. Three constraints keep the choice from leaking. (1) The owner's per-node policy is held at the owner's `self` scope. (2) Each cohort receives only **its own projection** — which of that member's nodes are in its audience — and the policy never names one cohort to another. (3) A node receives only **its own allowlist**: a deny entry never reaches the node it denies, because it would name the cohort there. A cohort peer therefore never sees a denied node as custody `none` or `unknown` (CC 3.1.3.3); a UI MAY label a member's node the cohort already knows of as "not a holder here, by its owner's choice".

The node's role sets the default, and the allow list is a missing grant (normative). Every node a person owns **is one of that person's devices**: claiming it makes it an occurrence of their identity (CC 3.3.6), whatever it is for. What differs between nodes is only **what replicates to them**. A node is set up as **personal** by default, and a personal node's allow list is all of its owner's cohorts. A node set up as a **server** — managed, infrastructure, or serving a community or an affiliation — has an allow list of only the cohorts it was set up to serve, so the owner's `self` and `family` content does not follow them onto it: signing in at work, or claiming a work server, never copies a family's photos there. The setup flow asks which. The class is the `device_class` of the node's `identity_occurrence` (personal = `phone` | `laptop`; server class = `server` | `embedded`

| `service` | `agent`); the allow list, when the owner sets one, is the `cohorts` member of the owner's `consent:replication` grant for that node (CC 3.3.7) — not a separate policy row. With no `cohorts` member a personal node receives every cohort its owner is in and a server node receives no `self` or `family` content.

The allow list is not a second mechanism. Replication already resolves on five axes — **who sent it** (`attesting_key_id`, provenance), **what it is** (the dimension and its plane), **who may receive** (`cohort_scope` and its roster or owner's nodes), **under which grant** (the transmission principle: `consent:replication` and the content key grant), and **whom it is about** (`attested_key_id`, `subject_key_ids`) — and a flow proceeds only when it conforms on all of them. A node outside a cohort's allow list simply **holds no grant** for that cohort's rows: the owner's `consent:replication` grant for that node (CC 3.3.7, which CC 6.1.5.2 B2 already makes the pin authorization) does not name them, and the at-rest cascade wraps no content key to it. The existing grant check then withholds, on the sender's side and at the receiver's hold decision alike, with no new gate.

Two classes of `self` row reach a node **regardless of its allow list**, each on an axis that already exists, and together they are what "the requisite for that node" means:

- **Rows that originate on the node** — the sender axis. A row the node itself authored belongs there, and flows outward to the owner's personal nodes like any `self` row.
- **Rows that refer to the node** — the subject axis. A row whose `attested_key_id` or `subject_key_ids` names the node, or a grant issued `for` it — its owner-binding, its key record, its `config:*` rows (CC 3.4.5, written by the owner about the node), its consent grants, its storage budget, its revocation — is always served to it. This is what makes remote administration over the grammar work: an owner changes a server's configuration from their phone by writing one `self` row about that node, and the row arrives. It is the rule the identity planes already follow (a row about the peer is always served to the peer), stated for every `self` row.

Changing a node's role is an edit to its allow list: narrowing withdraws the grants and the node descends what it no longer has a grant for; widening issues them and anti-entropy fills it.

The target at every tier (normative). An item's target holder count is counted over the content's **audience** — the claimed nodes its scope entitles to hold it under their owners' choices, as above — and is capped by it.

- **Small audiences.** Where the audience is smaller than the owner's target, the target is **the audience**: every audience node holds the item, and the replication factor is the audience size. The chunks are still sealed per CC 5.4.2. This is the case for most families.
- **Large audiences.** Where the audience is at least the target, the owner's target — or the cohort's default, which a node MUST be able to hold at — applies at every tier alike, and placement is rarest chunk first across the live cohort.
- **Every tier gets the same machinery.** Rarest-first placement, repair on loss, B2 pin-on-consent and B5 arbitration run at every tier. B3 budgets for `self` / `family` stay enforced locally only, as CC 6.1.5.2 requires. A person or family below its target sees it as custody (*here* on one device only), which is the signal to add a node, not a failure of the substrate.

What is unchanged. Consent is supreme at every tier: N5 forces descent of withdrawn or revoked content regardless of rarity, pin or target. A node never holds content it is not party

to. Durability therefore grows by a member **claiming another node** and admitting it to the cohort, never by placing a cohort's ciphertext on outsiders.

No holders outside the cohort. Sealed self/family chunks are not held on nodes outside the cohort, although such holders would see only indistinguishable ciphertext and durability would extend past the cohort: a holder learns that *some* blob of *some* cohort exists and when it arrived, which CC 5.2 withholds.

6.1.6 Deterministic ALM topology

The application-layer-multicast relay tree for large-N fan-out — the CC 5.3.3.2 "realtime large group (SFU/relay-tree)" profile. Determinism is the point: because every peer computes the same tree from the same inputs, no peer can quietly steer the tree to its advantage — but that same determinism turns one capacity lie into a universal one, so N8 hardens the inputs.

- `compute_alm_topology(snapshot)` → `topology` **is PIN-NORMATIVE as a contract** — a **pure, deterministic, integer-only** function (no IEEE-754; no `HashMap` iteration order) over `(capacity_ads, trust_grants, reachability_observations, locality)`, with specified lexicographic tie-breaks and canonical output order, such that **byte-equal inputs yield byte-equal output** across implementations. The byte-exact output is gated on the CC 6.1.4 vectors (incl. permutation-invariance cases) — **not** transcribed from the algorithm body as the source of truth.
- **N8 (capacity authenticity)** — capacity advertisements feeding the topology MUST be hybrid-verified (`SignedRelayCapacity`, domain `b"CIRISALM-CAPv2\0\0"`) **before scoring**; self-asserted `uplink_mbps` MUST NOT be the dominant, unbounded selection term (cap per steward-bound identity or make it throughput-challengeable). Determinism amplifies one capacity lie into a *universal* eclipse — "verified by an unspecified upstream tier" is not a guarantee.
- **D6 preserved** — `reachability_observations` are **ephemeral planner inputs**; they MUST NOT become attested, replicated, or witness-leafed state (CC 5.3.3.4 "reachability is never trust"). Resolution authority stays in CC 4.4.3.2.4.1; the topology consumes it, does not replace it. The determinism comparator reconciles to the CC 4.4.3.2.4.1 / CC 5.3.2.3 comparator family.
- **PIN line** — the topology *function contract* + `SignedRelayCapacity` / `SubStreamCommitment` signed preimages = **PIN-NORMATIVE**; **ALM parent-selection** (over per-peer RTT) = **edge-internal**.

6.1.7 Fail-secure summary (normative)

The mechanisms of this Part compose into one fail-secure posture. They are blind to the Anonymous Tier (WW-2, RB-1); subordinate to the consent and revocation model (N5, and the witness's deference to CC 5.3.2.3); gated by steward binding and founder quorum (N1) and the chain-length and cycle budget (N2); verified post-quantum at the gate (CC 6.1.3, N3, N8); and moved only by a plane that relieves and never expands — the inventory and the census measure, the mesh config relieves, the owner sets durability per item (CC 6.1.5.1). These invariants are the conformance target regardless of implementation timing.

6.1.8 Transitive trust discovery: not membership

`recursive_trust_bootstrap(SignedClaim, TrustGraph, WitnessChain)` → `verdict` lets a peer discover transitive trust by walking a signed witness chain to a root in its own trust graph. **It is reachability discovery beneath CEG's authority layer, not an admission shortcut** — discovering that you *can* reach someone is held strictly distinct from being *admitted* among them (Justice).

- **N1 (trust ≠ membership)** — a successful chain walk yields **trust+serve standing only** (CC 3.2 TRUST≠MEMBERSHIP). Admission to any **non-infrastructure** community remains gated, at the destination, by (a) the CC 3.2 **steward-binding** precondition (a live `user`-steward `delegates_to`, an admitted `identity_occurrence`) and (b) that community's `consensus_protocol`. `infrastructure` roots stay **founder-quorum**-gated; a transitive chain MUST NOT satisfy founder-quorum. `SignedClaim` carries the steward-binding fields so the gate is expressible.
- **N2 (self-supplied chains aren't evidence)** — the chain-length budget MUST be ≤ the CC 4.1.1 **5-hop cap**, trust-graph **cycles MUST be rejected** (CC 4.1.1), and the CC 4.1.1 **aggregate-weight cap** (default 0.5 × root_trust) MUST bound the standing one root confers transitively. A caller-supplied chain proves only its signatures, not a real lineage.
- **RB-1 (anonymous coexistence)** — anonymous-tier content MUST be ingestible / retainable / serveable with **no trust-graph position**; `recursive_trust_bootstrap` MUST NOT be required for, or invoked on, anonymous records.

6.1.9 The mesh census: returns, composites, and the base for governance (normative)

The census is how the mesh learns its own size and use without any node reading another's inventory. Each node sends one **census return** per period; two aggregators fold the returns of a period into one **mesh census** per class; the census is published as levels, never as changes; and no single aggregator can read a return, and only the two aggregators of a period acting together could. The census is measurement. It changes a node's behaviour only through the mesh-config plane (CC 6.1.5.1), it supplies the expected-active base that CC 4.5.13 counts governance against, and it carries a community's secret poll in the same shape.

Eligibility and the period. A return is sent once per period by a node whose owner binding chains to a bundle root (CC 3.2). Counts of voices — the expected-active base, a tally — count **owners**, never nodes; counts of the mesh — nodes, links, rows held — count nodes, at most eight per owner. The period is the global cadence of the constant tick (CC 5.4.5). A return is sent at an offset derived from `H(return_token ‖ period)`, at one fixed size, so its timing and size carry nothing about the node's activity; on a shaped link (CC 5.4.3) it replaces a cover frame, and on an unshaped link it shows only that the node participates, which every eligible node does.

The return is secret-shared (normative). Every field of a return is a one-hot vector over its log2 buckets: a count of 2 000 rows is a 1 in bucket 11 and a 0 elsewhere. The node splits each vector into two additive shares over a prime field and sends each share to a different aggregator, with a share of a fully linear validity proof that the two aggregators check jointly — every entry 0 or 1, the entries summing to 1 — without either learning the vector (the Prio3 histogram construction of the VDAF family). Neither aggregator sees any node's return; the sharing is information-theoretic, so value privacy rests on no post-quantum assumption, and the proof uses symmetric primitives only. A valid return adds exactly one count to exactly one bucket per field

— the stuffing ceiling, enforced rather than trusted. Each aggregator publishes a **partial**: the field-wise sum of the shares it accepted, the count of shares it summed, and a commitment over the accepted tokens. The **composite** is `partial_a + partial_b` element-wise in the field; each partial alone is uniformly distributed, so publishing both reveals nothing the composite does not. Because every accepted return from one owner contributes mass 1, `n_eff = source_count` by construction, and the dominance gate of CC 6.1.2.1.2 runs over the composite's members as a check on the aggregators. The multiplicity gate of CC 6.1.2.1 is a corpus-fold gate and does not read census returns or ballots; a census cell is gated by the k-floor (`source_count` $\geq$ `10`), one return per owner, the dominance gate and the ratchet, and identical returns are the ordinary case.

The rows. A return rides `census:share:{class}:v1`, one row per aggregator, **addressed**: a row whose signed `for_key_id` names one recipient is held by the author's node and the recipient's node and by no other; the serve gate sends it to the recipient only, no widening ever applies to it, and any other node offered it refuses it at its receive door as `federation_addressed_not_recipient`. It expires at the period's end and is reaped. The aggregators of a federation class are two serve seats per period; the partial is `census:partial:{class}:v1` and the composite `census:composite:{class}:v1`, both federation-scope rows. The composite is a CC 2.1 `scores` row whose signed members are the inputs of CC 6.1.2.1 — `version = 3`, `source_count`, `n_eff`, `max_source_multiplicity`, `mass_commitment_hex`, `member_commitment_hex`, `aggregation_algorithm_id` — verified at federation admission by the dominance gate; the tier object of CC 6.1.2.1 stays what it is for content composites. The member set, placement and types of every census member are pinned at CC 2.6.1.7; the families and their refusals are registered at CC 3.1.3. An aggregator MUST delete a share once it is summed into its partial, MUST hold no share past the period's end, and keeps partials and composites for a bounded number of periods per class.

What is published, and what is withheld (normative). A composite is a histogram per field per cell; a reader computes the median bucket and the trimmed statistics from it, and a relief reads the median. A published value is rounded to a grid of max(k, two significant figures). A cell with fewer than $\mathbf{k = 10}$ owners is merged upward and then withheld as *below the floor*; regions are self-declared and continent-sized at most, and are published only where the floor holds. A composite publishes levels, never deltas; it carries per-period tokens, never node ids; the directory holds no census data. Differential-privacy noise on small cells is an optional layer a deployment MAY add. **No return carries anything about the Anonymous Tier** (CC 5.4.6): there is no field for routes built, onion packets sent, relayed or received, or onion bytes; onion traffic is never a plane in a return; relay standing, hop-key publication and opt-out are returned only as counts per class per coarse region under the floor, which publishes nothing a reader could not compute from the public layer deal; onion **users** are never counted, in any form; and a node's liveness fold stays local. The availability answer a node shows its user reads the census relay composite and the node's own local liveness, never another node's return.

The base for governance, and the ratchet (normative). A community's expected-active count is a member-only composite. Each member's period return carries one cell per community it belongs to — the owner's own *recent* bit, set when the owner was active in this period or any of the previous three, and not something recoverable from a sum — sealed under the community's epoch key and sent, as `census:member:share:v1`, to two **members** of that community drawn from its roster by `H(group_exporter_secret ‖ period)`; a community cell never reaches a seat, because the pair (member, community) is a co-membership the shares cannot hide. The two publish `census:member:partial:v1` into the room, and `census:expected_active:v1` is the count of owners whose recent bit is set, recomputed by every member from the two partials; a row whose count or floor flag differs from a member's own recomputation is refused `census_`

`expected_active_mismatch`. The base CC 4.5.13 counts against is that count under a ratchet: it falls by at most a tenth per period (a larger fall is clipped to that tenth), and never below its **floor** — the standing roster at or below the count minimum of the CC 4.5.13 policy table, and above it the larger of the roster's rooted-active members and half of the last stable count. A fall of more than a third in one period is read as a partition: the base and every narrowing relief freeze until two consecutive periods each return at least two thirds of the pre-fall count. Silencing members therefore lowers a base slowly and in plain view, and never below what silencing cannot reach.

Polls, and what stays identified (normative). A governance act — an `objection:{state}` row, a roster change, a policy amendment under CC 4.5.13 — is an identified row counted against the census base; it is never secret. Separately, a community MAY hold a **secret poll or election** in the same instrument, at its own discretion: a member's community cell carries a one-hot over the poll's options plus *none*, the same two tally members fold it, and the result is `ballot:tally:{poll_id}:v1`, recomputed by every member; the dominance gate of CC 6.1.2.1.2 runs over the tally's members as over a composite, and one accepted cell per owner per period keeps `n_eff = source_count`. Every eligible member emits its cell every period, at the fixed size, whether or not a poll is open — an open poll fills a field that is otherwise padding — so the occurrence of a poll is invisible to an outsider, and its subject (`poll_id`) lives only in the sealed body. **Ballot secrecy among members is claimed**, against two non-colluding tally members. **Coercion resistance is not claimed**: a member can prove how they voted by revealing their shares. What a community does with a tally is the community's; a tally carries no standing of its own under Part 4.

What anyone holding the rows can check. (1) Both partials carry the same `member_commitment_hex` and `share_count`; (2) `composite = partial_a + partial_b` in the field; (3) every field's buckets sum to `source_count`, and `mass_total = source_count`; (4) every published cell has `source_count` $\geq$ `k`; (5) a contributor recomputes `member_commitment` over the published member list and compares. Until the seat order is a signed row, the pair of seats for a period is the sender's choice among seats and the door checks only that each recipient is a seat; the expected per-seat share of returns is then a statistical check, not an enforced one. Two colluding aggregators can drop returns or invent tokens up to the share they are expected to carry; the directory's bucketed count of live announced nodes bounds that to the rounding grid. The adversary rows for the census — stuffing, deanonymisation by small cells or differencing, a compelled aggregator, a global observer on return timing, a withholding adversary against a denominator, a poisoned aggregator — are at CC 1.13.3.3. The model is `formal/census_ballot`: share privacy while the two aggregators do not both collude, correctness, validity, agreement, one report per owner, the floor, the dominance gate, occurrence invisibility, the ratchet and the tally threshold, checked at nine instances with eight twins that fail where named, and the counting facts proved for any modulus. One reading the model pins: a stable period is one whose count meets the ratchet term, with the freeze released at two thirds of the pre-fall count.